

CAPSTONE PROJECT REPORT

Automate a Web Application Using Selenium WebDriver with Python

CAPSTONE ASSIGNMENT - 1

Student Name	Protyusha Mondal
Roll Number	12023052002072

ABSTRACT

Web application testing is an important part of software quality assurance because manual execution of repetitive workflows can be time-consuming and error-prone. This project implements an automated e-commerce testing workflow using Selenium WebDriver with Python and Pytest. The TutorialsNinja Demo application is used as the application under test. The automation launches Google Chrome, opens the application, performs login, searches for a product, selects the product, adds it to the shopping cart, updates the quantity, and verifies the cart contents. The implementation also reads test data from a JSON file, uses explicit waits to synchronize browser interactions, handles a possible cart notification and browser alert, captures screenshots, and generates an HTML execution report. The final execution recorded one passed test with zero failures, skips, or errors.

Keywords

Selenium WebDriver, Python, Pytest, Web Automation, E-Commerce Testing, JSON Test Data, HTML Test Report

1. INTRODUCTION

Automation testing allows software teams to execute repeatable test scenarios consistently. Selenium WebDriver is widely used for browser automation because it can control real browsers through programmatic commands. In this capstone, an e-commerce purchase scenario is automated from login through cart verification.

1.1 Project Scenario

The business scenario is a customer purchasing a product from an e-commerce website. The automated test represents a customer who logs in, searches for a MacBook, adds it to the cart, changes the quantity to two, and verifies that the product and quantity are correctly displayed in the cart.

1.2 Application Under Test

Application: TutorialsNinja Demo

Website: <https://tutorialsninja.com/demo/>

The application provides a suitable environment for demonstrating common e-commerce interactions such as account login, product search, product selection, and shopping-cart operations.

2. OBJECTIVES

- Automate an end-to-end e-commerce workflow using Selenium WebDriver with Python.
- Launch and interact with a web browser programmatically.
- Perform login using externally maintained test data.
- Search for and select a specified product.
- Add the selected product to the shopping cart.
- Update the product quantity in the cart.
- Verify the product and quantity using assertions.
- Handle a possible success notification and browser alert.
- Capture screenshots as test execution evidence and on failure.
- Generate an HTML test execution report using Pytest-HTML.

3. SCOPE OF THE PROJECT

In Scope	Description
Browser automation	Launching and controlling Google Chrome with Selenium WebDriver.
Authentication	Navigating to My Account → Login and entering credentials from JSON data.
Product search	Searching for the configured product name.
Cart operations	Adding the product and changing its quantity.
Validation	Checking the expected quantity and product name in the cart.
Exception evidence	Saving a failure screenshot if an exception occurs.
Popup/alert handling	Checking for a cart success notification and browser alert.
Reporting	Generating an HTML report using pytest-html.

4. TECHNOLOGIES AND TOOLS USED

Technology / Tool	Purpose
Python	Programming language used to implement the automation.
Selenium WebDriver	Automates browser actions and locates web elements.
Pytest	Test framework used to execute and validate the test case.

pytest-html	Generates an HTML execution report.
JSON	Stores reusable test data such as URL, credentials, product and quantity.
Google Chrome	Browser used for the automated execution.
WebDriverWait / Expected Conditions	Synchronizes the test with the web application's state.
VS Code / Terminal	Development and test execution environment.

5. PROJECT STRUCTURE

```

selenium-main/
├── 02_Capstone_Project/
│   ├── Source_Code/
│   │   ├── test_ecommerce.py
│   │   ├── testdata.json
│   │   └── requirements.txt
│   ├── Screenshots/
│   │   ├── cart_verification.png
│   │   └── test_failure.png
│   ├── Outputs/
│   │   └── report.html
│   └── Demo_Video.txt

```

5.1 File Description

File / Folder	Purpose
test_ecommerce.py	Main Selenium/Pytest automation script.
testdata.json	External JSON file containing the test URL, credentials, product and quantity.
requirements.txt	Python package dependencies required for the project.
Screenshots/	Stores screenshots captured during execution.
Outputs/report.html	Pytest-HTML execution report.
Demo_Video.txt	Contains the demonstration video reference.

6. AUTOMATION METHODOLOGY

The automation follows a sequential workflow corresponding to the capstone business scenario.

1. Load the JSON test data before test execution.
2. Create the required screenshots and reports directories if they do not already exist.
3. Launch Google Chrome using Selenium WebDriver and maximize the browser window.
4. Open the TutorialsNinja Demo application using the configured URL.
5. Navigate through My Account to the Login page.
6. Enter the username and password obtained from the JSON file and submit the login form.

7. Locate the search box, enter the configured product name, and submit the search.
8. Select the matching product from the search results and add it to the cart.
9. Check whether the cart success notification is displayed; if it is absent, continue the workflow.
10. Open the shopping cart and replace the current quantity with the configured quantity.
11. Click the cart update control.
12. Verify the updated quantity using an assertion.
13. Verify that the expected product is present in the cart.
14. Capture a screenshot of the verified cart.
15. Check for a browser alert and accept it when present.
16. If an unexpected exception occurs, capture a failure screenshot and re-raise the exception.
17. Close the browser in the finally block.

7. IMPLEMENTATION DETAILS

7.1 Loading Test Data

The automation separates test values from the Selenium logic by reading a JSON file. The configured test scenario uses the TutorialsNinja URL, a login account, the product MacBook, and a quantity of 2. Credentials are not reproduced in this report.

```
with open("test_data/testdata.json", "r") as file:  
    test_data = json.load(file)
```

7.2 Browser Launch and Synchronization

The script creates a Chrome WebDriver instance and a WebDriverWait object with a 15-second timeout. Explicit waits are used with expected conditions such as element-to-be-clickable and visibility-of-element-located. This reduces timing-related failures compared with immediately interacting with elements.

```
driver = webdriver.Chrome()  
wait = WebDriverWait(driver, 15)  
driver.maximize_window()
```

7.3 Login Automation

The test opens My Account, selects Login, locates the email and password fields, enters the values from the JSON file, and clicks the Login button.

7.4 Product Search and Cart

The product name is entered into the search field and submitted with the Enter key. The matching product is then selected and added using the Add to Cart control.

7.5 Quantity Update and Verification

After opening the cart, the existing quantity field is selected and replaced with the configured quantity. The update control is clicked. The script then reads the displayed quantity and verifies it against the expected value.

```
actual_quantity = updated_quantity.get_attribute("value")
assert actual_quantity == str(test_data["quantity"])
```

7.6 Popup and Alert Handling

The implementation checks for the application's success notification after adding the product. It also checks for a browser alert using `alert_is_present()`. If no alert is present within the short wait period, the test continues without failing.

7.7 Screenshot and Failure Handling

A screenshot named `cart_verification.png` is captured after successful cart verification. If an unexpected exception occurs, the exception handler saves `test_failure.png` before re-raising the error.

8. TEST CASES

ID	Test Scenario	Action	Expected Result	Observed Result
TC-01	Launch application	Open TutorialsNinja Demo	Application loads	Pass
TC-02	Login	Enter valid configured credentials	User reaches authenticated flow	Pass
TC-03	Search product	Search for MacBook	Matching product is displayed	Pass
TC-04	Add to cart	Click Add to Cart	Product is added	Pass
TC-05	Update quantity	Set quantity to 2 and update	Cart quantity becomes 2	Pass
TC-06	Verify product	Check product name in cart	MacBook is present	Pass
TC-07	Capture evidence	Save cart screenshot	Screenshot created	Pass
TC-08	Handle alert	Check for browser alert	Alert accepted if present; otherwise test continues	Pass
TC-09	Generate report	Run Pytest with pytest-html	HTML report generated	Pass

9. TEST EXECUTION AND RESULTS

The supplied HTML execution report was generated by `pytest-html` on 23 September 2026 at 01:38:31. The report records one executed test and shows 0 failed, 1 passed, 0 skipped, 0 expected failures, 0 unexpected passes, 0 errors, 0 reruns, and 0 retried tests. The recorded duration was approximately 18 seconds.

Metric	Result
Total tests	1

Passed	1
Failed	0
Skipped	0
Errors	0
Reruns	0
Retried	0
Execution duration	Approximately 18 seconds
Report format	HTML (pytest-html 4.2.0)

10. TEST EXECUTION EVIDENCE

10.1 Cart Verification Screenshot

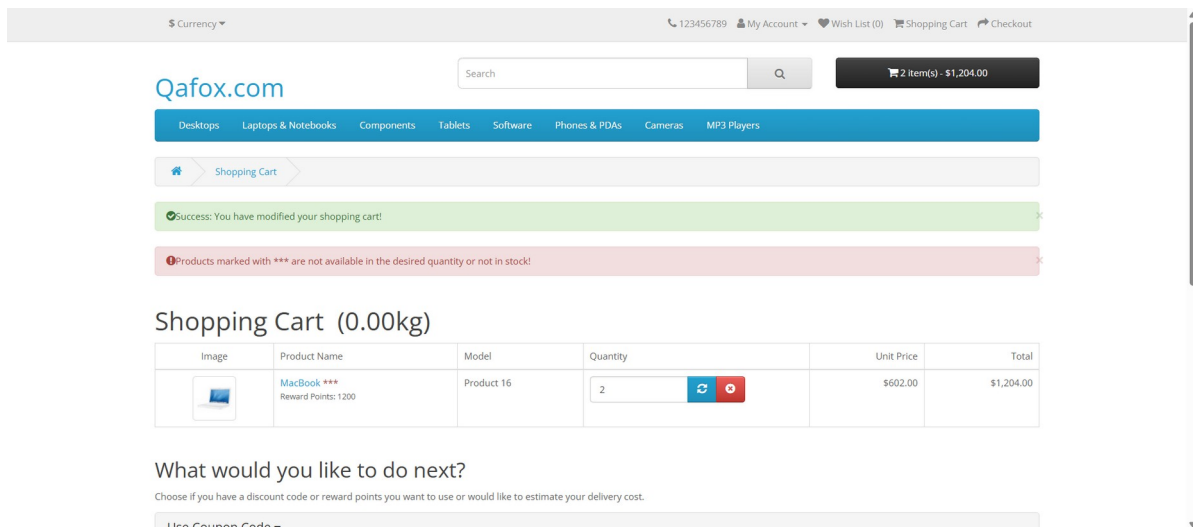


Figure 1: Cart verification captured during the successful test execution.

10.2 Failure Screenshot Artifact

The project also contains a failure screenshot artifact. The automation is designed to overwrite/save a failure screenshot when an unexpected exception occurs, providing visual evidence for troubleshooting.

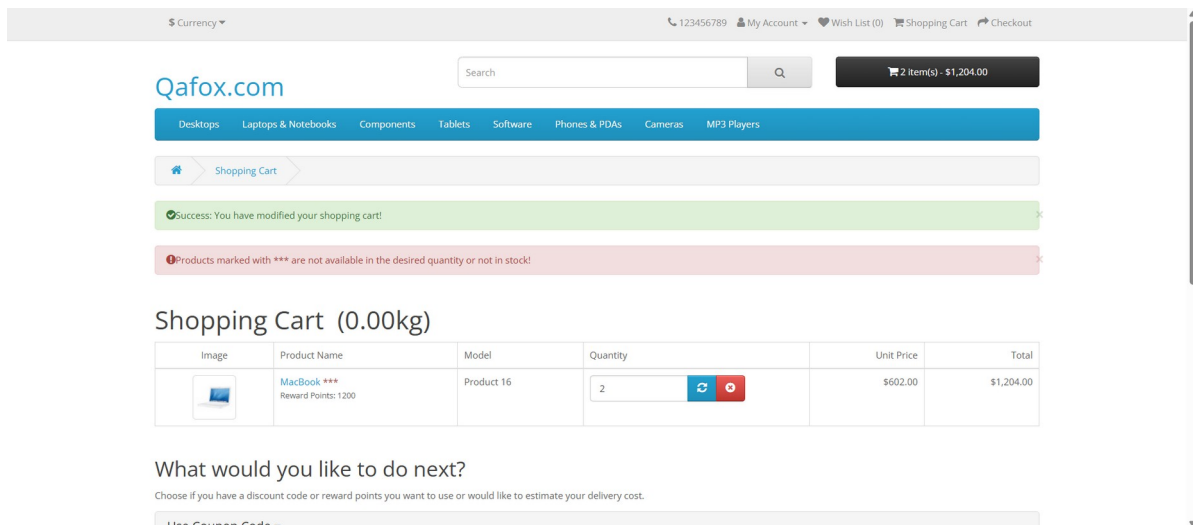


Figure 2: Failure-screenshot artifact included with the project.

11. HTML EXECUTION REPORT

The project uses pytest-html to create a self-contained HTML report. This report provides a browser-readable summary of the test run and its result. The generated report is stored under the Outputs directory as report.html.

```
python -m pytest test_ecommerce.py -v --html=../Outputs/report.html --self-contained-html
```

The successful execution recorded 1 passed test and no failures or errors.

12. REQUIREMENTS AND EXECUTION

The project requirements file specifies the Python packages required for the Selenium automation. The typical execution sequence is:

18. Create and activate a Python virtual environment.
19. Install the packages listed in requirements.txt.
20. Navigate to the Source_Code directory.
21. Run the Pytest test file.
22. Optionally generate the self-contained HTML report using pytest-html.

```
python -m venv venv
# Windows PowerShell
.\venv\Scripts\Activate.ps1
```

```
pip install -r requirements.txt
```

```
python -m pytest test_ecommerce.py -v
```

```
python -m pytest test_ecommerce.py -v ^
--html=../Outputs/report.html --self-contained-html
```

The final command may be written as a single line in Windows PowerShell if required.

13. LIMITATIONS

- The automation covers the specific e-commerce workflow required for the capstone scenario rather than the complete website.
- The test depends on the availability and current structure of the TutorialsNinja Demo application.
- The current implementation uses a single Pytest test function and one JSON test-data scenario.
- The implementation is a direct Selenium test script and does not claim to implement a separate Page Object Model architecture.
- The test credentials should be kept private and should not be committed to a public repository.

14. CONCLUSION

The project successfully demonstrates browser-based e-commerce automation using Selenium WebDriver with Python and Pytest. The automated workflow covers the required business scenario from application launch and login through product search, cart operations, verification, screenshot capture, popup/alert handling, and report generation. External JSON test data improves separation between test values and automation logic, while explicit waits improve synchronization with the web application. The final supplied execution report confirms one passed test with zero failures and errors. Overall, the project provides a practical demonstration of automated functional testing for a web application.

15. FUTURE SCOPE

- Add multiple data-driven test scenarios using parameterized test data.
- Separate page interactions into reusable Page Object classes for larger test suites.
- Add more negative and validation scenarios such as invalid login and unavailable products.
- Integrate the test suite with a CI/CD pipeline.
- Add richer reporting with screenshots embedded directly in test results.

16. REFERENCES

- Selenium WebDriver documentation: <https://www.selenium.dev/documentation/>
- Pytest documentation: <https://docs.pytest.org/>
- pytest-html documentation: <https://pytest-html.readthedocs.io/>
- TutorialsNinja Demo application: <https://tutorialsninja.com/demo/>
- Python documentation: <https://docs.python.org/>